

AEVUM

THE DEFINITIVE FINAL BLUEPRINT

Deterministic Vulnerability Remediation · Proof-as-a-Service · Enterprise Security Platform

4,000→4

REAL VULNS

97%

MTTR REDUCTION

72Cr

YEAR 3 ARR

9

BUILD LAYERS

Overview & Core Proposition

The world's first deterministic vulnerability remediation platform. Not detection — confirmation, root-cause tracing, sandbox simulation, and cryptographic proof.

4 Real CONFIRMED VULNERABILITIES	843 FALSE POSITIVES ELIMINATED	97% MTTR REDUCTION	■ 72Cr TARGET ARR YEAR 3
--	--	--	--

The Problem Aevum Solves

■	Snyk reports 4,000 alerts. Which 4 are real? Your engineers spend months triaging. 3,996 are false positives. Zero way to know which without Aevum.
■	Maven BOM precedence is invisible to every scanner The version Snyk flags as vulnerable may not even be on your runtime classpath. Different BOM wins. Completely different version ships.
■	SOC2 auditors want proof — not PDFs Snyk gives you a report. Aevum gives cryptographic evidence: before classpath, after classpath, build log, test results, SHA-256 hash.

The 5 Unfair Advantages

01	Deterministic Resolution Engine 2+ years of Maven/Gradle BOM archaeology. Snyk uses ML confidence. Aevum uses exact BOM precedence calculation. Different DNA — impossible to copy.
02	Proof-as-a-Service Cryptographic evidence package per fix: before/after classpath, build log, test results, SHA-256. Auditors accept this. Competitors would need full architecture rebuild.
03	Precedent Memory Network Effect Fix from Customer A improves suggestions for Customer B (anonymised). Data moat compounds. The longer customers use Aevum, the smarter it gets.
04	Open-Core Flywheel Free CLI creates 100,000+ developer advocates who pull enterprise sales upward. Competitors charge for everything. Aevum gives away the wedge.

05

Value-Based Pricing

Charged for MTTR reduction, not seats. When customer saves █2Cr in breach risk, they don't blink at █20L/year.

Complete Technology Decisions

Every choice justified. No over-engineering. No under-building.

Backend Microservices — Java 21 / Spring Boot 3.x

SERVICE	TECHNOLOGY	SCALES WITH
aezum-core	Java 21 Library + Maven Embedder + GraalVM	N/A — domain library
aezum-cli	Java 21 + GraalVM Native Image	N/A — single binary, 18ms startup
aezum-control-plane	Spring Boot 3 + PostgreSQL + Redis	Tenant count
aezum-signal-ingestion	Spring Boot + Kafka producers	Scan frequency
aezum-verification-pipeline	Spring Boot + Virtual Threads (Java 21)	Alert volume — 50K concurrent
aezum-remediation-fabric	Spring Boot + Docker-in-Docker	Fix requests
aezum-delivery-orchestrator	Spring Boot + GitHub API	PR volume
aezum-observability	Spring Boot + ClickHouse + Grafana	Data volume
aezum-ai-engine	Java + Python FastAPI + PyTorch	Model complexity
aezum-ares	Spring Boot + Claude API + Phantombuster	LinkedIn lead pipeline

Data Layer

DATABASE	ROLE	WHY
PostgreSQL 16	Primary datastore	RLS from Day 1. PgBouncer pool. pgvector replaces Pinecone.
Neo4j / Neptune	Dependency graph	(:Artifact)-[:DEPENDS_ON]->(:Artifact). BOM trace queries.
Redis Cluster	Cache + JWT	Maven resolution L2 cache. 94% hit rate. 24h TTL.
ClickHouse	Analytics engine	Billions of vuln events. Real-time MTTR. 100x faster than Postgres.
Apache Kafka	Event backbone	14 topics. Exactly-once. KEDA autoscale off consumer lag.
S3 / MinIO	Evidence locker	Proof packages. Append-only. Never deleted. SOC2 evidence.

Infrastructure

LAYER	TOOL	CONFIG
Container orchestration	Kubernetes EKS	KEDA autoscaling. HPA off Kafka consumer lag.
API Gateway	Kong	Rate limiting, JWT validation, OpenAPI routing.
Service mesh	Istio	mTLS between all services. Zero-trust network.
Secrets	HashiCorp Vault	Auto-rotated. Never in env vars or config files.
IaC	Terraform	Multi-region: EU + US + APAC. Day 1 configuration.

LAYER	TOOL	CONFIG
CI/CD	GitHub Actions	Test → Self-scan → Build → Stage → Blue-green prod.
DNS + DDoS	Cloudflare	WAF + CDN. Sub-100ms global latency.
Payments	Stripe	Subscriptions + usage-based. Webhook to billing service.

System Design & Build Sequence

9 layers. Never build layer N+1 before layer N is validated. Each gate protects you from building the wrong thing at scale.

The 9 Build Layers — In Exact Order

LAYE R 0	Core Domain Engine MavenBomResolver + 6-stage VerificationPipeline. Everything else wraps this. The crown jewel.
LAYE R 1	aevum-cli (OSS) GraalVM native binary. scan + trace + apply + simulate commands. Validates engine on real codebases.
LAYE R 2	Control Plane Auth/JWT, multi-tenancy (PostgreSQL RLS), RBAC, Stripe billing. Every cloud feature needs this.
LAYE R 3	Signal Ingestion + Verification Pipeline Kafka backbone. Scanner webhooks in → confirmed vulns out. Cloud version of CLI engine at scale.
LAYE R 4	Remediation Fabric + Delivery Orchestrator Fix generation, Docker sandbox simulation, GitHub PR with cryptographic proof. The actual product.
LAYE R 5	Web Application (React) CISO dashboard, repo health, evidence viewer, live activity feed, MTTR trends.
LAYE R 6	AI / ML Layer Claude API for explanations + PR descriptions. Python PyTorch for 90-day CVE prediction.
LAYE R 7	Observability + Compliance MTTR metrics, SOC2/NIST/ISO auto-mapping, ClickHouse analytics, executive dashboards.
LAYE R 8	ARES Marketing Agent LinkedIn monitor → Claude classifier → DM sequencer → HubSpot CRM → Calendly demo booking.
LAYE R 9	Enterprise Features Self-hosted VPC, multi-region, marketplace listings (AWS/Azure/GCP), white-label OEM.

The 6-Stage Verification Pipeline

STAGE 01	Signal Normalizer Normalise all scanner formats (Snyk JSON, Trivy SARIF, BlackDuck XML) into unified VulnerabilitySignal. Strip duplicates via SHA-256 hash.
STAGE 02	Effective Version Checker Is the version scanner reported actually what Maven selected? Run BOM precedence resolution. 85% of false positives eliminated HERE .
STAGE 03	Classpath Presence Verifier Even if version matches, is it on the RUNTIME classpath? Check /WEB-INF/lib, fat-JAR manifest, module path.
STAGE 04	Runtime Reachability Analyser Is the vulnerable code path actually called? Static call-graph from app entry points to vulnerable classes.
STAGE 05	Exploitability Assessor EPSS score (exploitation probability 30 days) + KEV (Known Exploited Vulnerabilities) + network exposure context.
STAGE 06	Confidence Scorer Final 0–100 score. <70 = probable false positive. >90 = act immediately. Attached to immutable proof package.

Kafka Topics

TOPIC	PARTITIONS	RETENTION	PURPOSE
aezum.signals.raw	12	7 days	Raw scanner webhook payloads
aezum.signals.normalized	12	30 days	Unified signal format, deduplicated
aezum.verification.requests	24	7 days	High parallelism — more partitions = more throughput
aezum.verification.results	12	90 days	Confirmed / false positive results
aezum.remediation.requests	12	7 days	Confirmed vulns waiting for fix generation
aezum.delivery.requests	6	7 days	Simulation-passed PRs ready to ship
aezum.audit.events	6	365 days	IMMUTABLE compliance trail — compacted, never deleted

52-Week Execution Sequence

Every week has one primary output. Build. Validate. Gate. Repeat.

PHASE 0 Core Engine + CLI Weeks 1–8

Ship aevum-cli v1.0. Get 1,000 developers to say 'holy sh*t.'

Week 1–2

aevum-core: Domain model + Maven BOM Resolver v0.1

Artifact, BomDeclaration, EffectivePom. Maven Embedder integration. BomPrecedenceCalculator. 4 precedence rules implemented.

Week 3–4

aevum-core: 6-stage VerificationPipeline

All 6 stages. Unit tests with real-world CVE fixtures. Accuracy target: 95%+ false positive detection on test suite.

Week 5–6

aevum-cli: scan + trace + Snyk/Trivy adapters

GraalVM native build. Single binary no JVM. aevum scan --scanner=snyk. Rich terminal output with stage progress.

Week 7–8

aevum-cli: apply + simulate. GraalVM v1.0 release

Fix application. Docker sandbox simulation. aevum apply --simulate=true. Target: 18ms cold start.

GATE: 1,000 CLI downloads. 500 GitHub stars. 10 enterprise inquiries. Resolver accuracy >95% on test suite.

PHASE 1 Cloud MVP + Design Partners Weeks 9–20

3 paying design partners. Prove MTTR reduction with real data. First invoices.

Week 9–10

Hacker News: 'Show HN: I built a tool that confirms if Snyk alerts are real'

Post, respond to every comment. Collect 50+ GitHub issues. Fix accuracy problems from real codebase feedback.

Week 11–12

aevum-control-plane: Auth + JWT + multi-tenancy

Spring Security, OAuth2 (GitHub, Google), API keys for CLI. PostgreSQL RLS. PgBouncer setup.

Week 13–14

aevum-signal-ingestion: Kafka + scanner webhooks

Snyk, Trivy, OWASP webhook receivers. Normalization service. Topic creation. CVE enrichment (NVD + EPSS + KEV).

Week 15–16

aevum-verification-pipeline: Cloud engine

Kafka consumer. Maven resolver with Redis caching. Virtual threads config. Neo4j graph writes.

Week 17–18

aevum-remediation-fabric: Fix gen + Docker sandbox

BomOverrideStrategy, ExclusionStrategy, VersionBumpStrategy. DockerSandboxService. Pre-warmed pool.

Week 19–20

aevum-delivery-orchestrator: GitHub App + PR automation

Branch creation. ProofPackageBuilder. Evidence S3 upload. Full audit PR body auto-generated by Claude.

GATE: 3 paying pilots at █2K–5K/month. 50% MTTR reduction proven with real customer data.

PHASE 2 Product-Market Fit Weeks 21–36

24 paying customers. █2L MRR. Web dashboard live. SOC2 audit started.

Week 21–22

aevum-web: React dashboard MVP

Confirmed vulns list, repo health, live activity feed, proof package viewer. WebSocket real-time updates.

Week 23–24

aevum-ai-engine: Claude integration

Natural language CVE explanations. AI PR descriptions. Policy compliance assessment via Claude Sonnet.

Week 25–26

Stripe billing + team invites + onboarding wizard

Subscription plans. Seat management. GitHub App installation wizard. 14-day free trial flow.

Week 27–28

aevum-observability: MTTR dashboard + compliance export

ClickHouse analytics. SOC2 evidence generation. NIST/ISO mapping. CISO executive view.

Week 29–30

aevum-ares: LinkedIn agent v1 + HubSpot CRM

Phantombuster monitoring. Claude signal classifier. 3-stage DM sequencer. Calendly integration.

Week 31–32

SOC2 Type II audit prep + pen test + bug bounty

Required to close enterprise deals. Launch bug bounty on HackerOne. External pen test firm.

Week 33–36

Gradle support + npm adapter + Jira integration

Expand beyond Maven. 3x addressable market. Jira bi-directional sync for enterprise workflows.

GATE: 24 customers. █2L MRR. 80%+ retention. NPS 50+. SOC2 audit in progress.

PHASE 3 Scale + Enterprise Weeks 37–52

100 customers. █10L MRR. First Fortune 500 logo.

Week 37–40

Self-hosted VPC + enterprise SSO (SAML/SCIM)

Required for banks, healthcare, defence. Helm charts. Okta, Azure AD, Ping Federation support.

Week 41–44

ML/AI engine: predictive vulnerability model

PyTorch: commit velocity + CVE history → 90-day CVE probability score. Shift from reactive to predictive.

Week 45–48

Multi-region: EU + APAC + data residency routing

GDPR compliance. EU data stays in eu-west-1. APAC in ap-southeast-1. DataResidencyRouter service.

Week 49–52

AWS + Azure + GCP marketplace listings

Private offers for enterprise. Marketplace handles invoicing. Reduces procurement friction by 80%.

GATE: 100 customers. ■10L MRR. 2x expansion revenue. Series A ready. 5 Fortune 500 logos.

Maximum Performance Architecture

Aevum scans itself in every CI pipeline. Every target is measurable. Non-negotiable.

Performance Targets

OPERATION	TARGET	HOW	PRIORITY
CLI cold start	< 50ms	GraalVM native -O3, --gc=epsilon, --static	CRITICAL
Signal ingestion webhook	< 200ms	Kafka async — no blocking DB writes in hot path	CRITICAL
Maven BOM resolution (cached)	< 5ms	Redis L2 + Caffeine L1 in-memory cache	CRITICAL
Maven BOM resolution (fresh)	< 8 seconds	Virtual threads + Maven Embedder parallel	HIGH
6-stage verification (full)	< 30 seconds	Stages 1–3 parallel where possible	HIGH
Fix simulation (Docker sandbox)	< 3 minutes	Pre-warmed Docker pool, parallel Maven builds	HIGH
PR creation with proof	< 10 seconds	GitHub API async, S3 pre-signed URLs	MEDIUM
CISO dashboard load	< 1 second	ClickHouse materialised views + Cloudflare CDN	HIGH
END-TO-END: signal → merged PR	5 MINUTES	All of the above combined. The headline metric	CRITICAL

Java 21 Virtual Threads — The Core Performance Advantage

This is why Aevum uses Java, not Node.js or Python. Virtual threads allow 50,000 concurrent Maven resolutions on 8 CPU cores.

APPROACH	CONCURRENT VERIFICATIONS	HARDWARE	HOURLY AWS COST
Platform threads (old Java)	500 max	c5.2xlarge × 1	~\$0.38/hr
Virtual threads (Java 21)	50,000	c5.2xlarge × 1	~\$0.38/hr
Matching VT capacity without VT	50,000	c5.2xlarge × 100	~\$38/hr

Maven Resolution 3-Layer Cache

Cache key = SHA-256 of pom.xml content. If pom.xml unchanged, resolution result is identical. Real-world cache hit rate: ~94%.

CACHE LAYER	SPEED	CAPACITY	TTL
L1: Caffeine (in-process)	~0.001ms	1 GB JVM heap	Until service restart
L2: Redis Cluster (shared)	~2ms	32 GB dedicated	24 hours
L3: Full Maven resolution (fallback)	< 8 seconds	On-demand compute	No cache — compute fresh

Self-Healing CI — Aevum Scans Itself

Aevum runs on its own codebase in every push. Publish results at [aevum.dev/dogfood](#) — your strongest enterprise sales proof point.

CI STEP	COMMAND	PURPOSE
Self-scan	aevum scan --scanner=trivy --scanner=owasp-dc	Aevum finds its own confirmed vulnerabilities
Auto-fix	aevum apply --simulate=true --auto-pr=true --min-confidence=90	Order PRs with proof for real confirmed vulns
Validation	./mvnw verify -pl aevum-core	Confirms fix didn't break Aevum itself
Publish	aevum report --public=true	Live dashboard at aevum.dev/dogfood

JVM Flags for Verification Pipeline

FLAG	VALUE	WHY
-XX:+UseZGC	enabled	Zero stop-the-world GC pauses. 10ms max pause budget.
-Xms / -Xmx	4g / 16g	Pre-allocate 4GB. Allow burst to 16GB during mass scans.
-XX:+AlwaysPreTouch	enabled	Pre-allocate heap on startup. No lag on first scan batch.
-XX:+UseStringDeduplication	enabled	Maven artifact coordinate strings deduplicated in heap.
-Xss	512k	Tight stack per virtual thread. More threads per GB RAM.
-XX:ParallelGCThreads	8	All CPU cores used for GC. Never starves application.

Autonomous Revenue Engine System

Fills your calendar with qualified enterprise demos 7 days a week while you build. Precision targeting — not spam.

ARES Daily Operating Rhythm

TIME	ACTION	DETAIL	VOLUME
06:00	SCRAPE	Phantombuster pulls LinkedIn posts matching 14 keywords	500 posts/day
07:00	CLASSIFY	Claude API scores each post 1–10 for buying intent 7+ = engage queue	
08:00	COMMENT	High-quality insight comments — never promotional	5–8 comments/day
10:00	CONNECT	Connection requests to 48h signal matches with notes	10 requests/day
14:00	DM QUEUE	Follow-up messages in correct sequence stage	3–5 DMs/day
17:00	REVIEW	Human reviews 9–10 scored leads personally	Top priority leads
19:00	ANALYTICS	Response rates, demos booked, HubSpot CRM updated	Daily dashboard

14 Buying Signal Keywords ARES Monitors

■ Maven BOM conflict	■ Snyk alert fatigue
■ CVE remediation	■ DevSecOps hiring
■ AppSec	■ Spring Boot security
■ vulnerability triage	■ SOC 2 audit deadline
■ SBOM compliance	■ Log4Shell
■ dependency hell	■ false positive CVE
■ MTTR security	■ compliance deadline

The 3 DM Stages — Exact Copy

DAY 0 — Connection Note (300 chars max)

Saw your post about [Maven/Snyk/CVE]. We built something that confirms which alerts are actually real before you spend time on them. Building in this space — would love to connect and share what we learned.

DAY 2 — First DM (Value. Zero ask.)

Hey [Name], thanks for connecting.

That post about [Snyk alerts / Maven hell] resonated — we had the same problem before building Aevum.

Quick insight: The reason most alerts are false positives is Maven's dependency mediation algorithm. What Snyk reports as vulnerable isn't always what ships to production.

I wrote a breakdown here: [blog link]. No ask — just thought it'd be useful.

DAY 10 – The Close

[Name] — last message, I promise.

If vulnerability noise is still a pain point, I'll give you a 30-minute live session where I run Aevum against one of your actual repos and show you the confirmed vs unconfirmed split in real time.

If it doesn't blow you away, no hard feelings. [Calendly link]

ARES Tech Stack

COMPONENT	TOOL	NOTES
LinkedIn scraper	Phantombuster API	50 posts/day safe limit. Never gets account flagged.
Signal classifier	Claude Sonnet API	Scores 1–10. JSON response. Spring Boot calls API.
CRM sync	HubSpot Starter	Lead records, pipeline stages, activity log.
Message generator	Claude Sonnet API	Personalised DMs using their post content + company.
Demo scheduling	Calendly API	Auto-embedded in Day 10 DM. Confirmed → Slack notif.
Safety throttle	Custom rate limiter	Max 20 LinkedIn actions/day. No account risk.

Pricing, Projections & Unit Economics

Value-based pricing. Customers pay for MTTR reduction, not seats.

Pricing Tiers

TIER	PRICE	TARGET	KEY FEATURES	REVENUE RANGE
CLI Free	0	Individual devs	Local scan, BOM trace, community support	Acquisition only
Team	8,000/dev/mo	50–500 devs, Series A–C	Cloud dashboard, GitHub auto-PR, 50 repos, 50k	50k–50K MRR/customer
Business	35,000/dev/mo	500–5,000 devs, regulated	Unlimited repos, SOC2 export, SSO/SAML, 99.99%	50K–5L MRR/customer
Enterprise	Custom	Fortune 500, banks	Self-hosted VPC, OEM, ML fine-tune, 99.99%	5L–50L ARR/customer

Financial Projections

METRIC	YEAR 1	YEAR 2	YEAR 3	YEAR 4	YEAR 5
Revenue	24L	1.2Cr	10Cr	30Cr	72Cr+
Customers	12	80	300	700	1,500+
ARPU/month	20K	12.5K	27K	35K	40K
Gross Margin	70%	78%	85%	87%	88%
Net Revenue Retention	105%	120%	140%	150%	155%
Headcount	1 (you)	5	25	80	200
Fundraise	Bootstrapped	1Cr Seed	5Cr Series A	20Cr Series B	IPO / M&A
EBITDA Margin	-100%	-20%	22%	42%	52%

The ROI Conversation That Closes Deals

Run this calculation live on the demo call using the customer's own numbers from their Snyk report:

VARIABLE	EXAMPLE VALUE	SOURCE
Engineer hourly cost	2,500	50L salary ÷ 2,000 hours/year
False positive alerts (monthly)	843	From their live Snyk scan — shown on screen
Hours per false positive triage	1.5 hours	Industry average (your research)
Monthly false positive cost	31,61,250	843 × 1.5 × 2,500
Aevum Business (20 devs)	7,00,000/month	35,000 × 20 developers
NET MONTHLY SAVING	24,61,250	Their money. Their numbers. Undeniable.
MONTH 1 ROI	452%	CFO approves in the same meeting.

Unit Economics

METRIC	YEAR 1	YEAR 2	YEAR 3
CAC (Direct PLG)	■0	■0	■0
CAC (Inside Sales)	■50K	■35K	■25K
CAC (Enterprise Field)	■5L	■4L	■3L
LTV:CAC Ratio	3:1	5:1	8:1
CAC Payback Period	12 months	8 months	5 months
Expansion Revenue %	10%	25%	40%

08 — GO-TO-MARKET

Three Channels. Sequential Activation.

Never run all three at once. Master one before activating the next.

Channel 1 — Developer PLG (Months 1–6)

Open-source CLI creates organic pull. Zero CAC. The foundation all other channels sit on.

- Ship aevum-cli v1.0 to GitHub. Make the output spectacular — one command that makes a developer stop and stare.
- Post 'Show HN: I built a tool that confirms if your Snyk alerts are real.' Respond to every comment.
- Write: 'Why 70% of your Snyk alerts are wrong' — SEO magnet + Hacker News bait.
- Write: 'How Maven BOM precedence actually works' — long-tail SEO for Java developers.
- Post daily on LinkedIn for 14 days. Only education. Zero promotion. Build authority first.
- DM 20 developers you already know. Ask: 'Would you try this free tool on your repo?'

Channel 2 — ARES Outbound (Months 3–12)

LinkedIn AI agent runs 7 days/week. Targets DevSecOps engineers, AppSec leads, VP Engineering at Series A–C Java shops.

- Configure ARES with 14 signal keywords. Start monitoring from Month 3 when CLI credibility is established.
- 5–8 insight comments/day. 10 connections/day. 3–5 DMs/day. Total: 20 LinkedIn actions/day maximum.
- Target ICP ONLY: Java/Spring Boot, 50–500 devs, Series A+, active Snyk user, compliance pressure.
- Human reviews every lead scored 9–10 personally. Templates handle everything below.
- Weekly target: 5 demo calls booked from ARES pipeline alone.

Channel 3 — Enterprise Inbound (Months 9+)

Every closed deal becomes a case study. Case studies become LinkedIn posts. Posts become inbound demos. The compounding flywheel.

- Publish: 'How [Company] cut MTTR from 60 days to 4 hours using Aevum' — with permission.
- Launch 'State of Vulnerability Remediation 2025' annual report. Gated lead magnet.
- Speak at RSA Conference, DevSecOps Days, SpringOne. Expertise pulls inbound.
- Public dogfood dashboard (aevum.dev/dogfood) — Aevum's own security posture. Live proof.
- Gartner + Forrester analyst briefings at Month 18. Category creation begins.

ICP Definition — Never Deviate Until ■10L MRR

09 — NON-NEGOTIABLE RULES

The Rules That Guarantee Success

Every rule exists because a founder learned it the hard way. None are optional.

01 Ship ugly. Ship early.

CLI v0.1 does not need to be perfect. It needs ONE output that makes a developer stop and stare. Ship in Week 6, not Month 6. 80% accuracy at launch is fine. Polish comes from real user feedback, not from building in isolation for 6 months.

02 Talk to 50 people before building the cloud product.

The React dashboard costs 3 months of engineering. Make sure 50 developers have confirmed they would pay for it before writing a single line of cloud code. Design partners do this for free.

03 Never pitch in the first message. Ever.

Every first touch — comment, connection note, DM — provides a real Maven/security insight. Zero promotional language. The moment you pitch first, the lead is gone. Insights build trust. Trust enables sales.

04 MTTR reduction is the only metric that matters for sales.

Not DAU. Not GitHub stars. Not NPS. MTTR reduction in hours, converted to rupees. Put this number in every conversation, every post, every email. It's what makes CFOs approve contracts without blinking.

05 One ICP until ■10L MRR. No exceptions.

Java/Spring Boot, 50–500 devs, Series A+, active Snyk, compliance pressure. Every deviation is a distraction. Every 'interesting exception' is a time sink. Focus is what makes ARES accurate and sales cycles short.

06 Never build layer N+1 before layer N is validated.

The build sequence is not a suggestion. If Maven resolver isn't 95% accurate, don't build the cloud pipeline. Each gate exists to protect you from building the wrong thing at scale. Discipline here = speed overall.

07 Dogfood from Day 1.

Run Aevum against Aevum in every CI push. Publish results publicly. A CISO visiting your site sees that Aevum's own codebase is clean and protected. It answers their question before they ask it.

08

Your Java + AI combination IS the moat.

Most founders pitch first, demo second. You can sit in front of a CTO, open a terminal, run a live scan on their actual codebase, and show them in 5 minutes what their █50L/year Snyk contract missed. No slide deck does this. Use it in every single demo.

90-Day Success Scorecard

METRIC	DAY 30	DAY 60	DAY 90
GitHub stars (aevum-cli)	100	300	600
LinkedIn followers	500	1,500	3,000
ARES comments posted	120	300	500
Demo calls booked	10	30	60
Design partners active	3	7	10
Paying customers	0	2	5
MRR	█0	█1L	█3L
Maven resolver accuracy	90%	95%	98%
False positive elimination rate	85%	92%	97%

THE NORTH STAR

"The market does not need another security startup. It needs a founder who can sit in front of a CTO, run a live scan on their actual codebase, and show them in 5 minutes what their ■50L/year Snyk contract missed."

That founder is you. Java + AI skills. You can build every layer yourself, demo Aevum live on real repos in real time, and explain Maven BOM precedence technically in a way that builds instant credibility with every CTO and CISO you meet. ARES fills your calendar. You close the deals. That is the entire plan.

- LAYER 0–1 Core engine + CLI. Ship Week 6.
- LAYER 2–4 Cloud pipeline. Design partners. Month 4.
- LAYER 5–7 Dashboard + AI + Observability. ■2L MRR. Month 8.
- LAYER 8 ARES agent live. Pipeline flowing. Month 10.
- LAYER 9 Enterprise. Self-host. Fortune 500. Month 18.